AIQA — Phase 4 API Testing Implementation Report

Date: 2026-08-25 **Scope:** Implement a dedicated API Testing page — real request builder, real HTTP execution, real assertions, real evidence — reusing existing executor/compiler/persistence architecture wherever it genuinely exists. **Report generated by:** Claude (Sonnet 5), in-session implementation + live verification against a locally running orchestrator, including a real browser round-trip and a self-verifying live write (the API Testing page creating a real project via its own real HTTP request).

---

1. Executive Summary

Phase 4 replaces `APITestingPage.tsx`'s previous honest-but-inert redirect ("real API testing runs through the Execution Hub, this page has no request-builder UI of its own") with a genuinely real, dedicated request builder. The fresh baseline audit found something decisive: Phase 2.1 had already built almost everything Phase 4 needed — a closed `ExecutionPlan`/`Action` schema (`REQUEST`/`ASSERT_STATUS`/`ASSERT_HEADER`/`ASSERT_BODY_FIELD`), a real SSRF/target guard (`execution_plan::validate_plan`), and a real, native, no-subprocess HTTP executor (`executor::CompiledApiExecutor`, direct `reqwest`, real assertion evaluation against the real response) — but it was only ever reachable by first compiling a persisted test case's natural-language steps. The one genuinely missing piece was a way to construct an `ExecutionPlan` **directly** from a user's own explicit request-builder input, bypassing compilation entirely because the user *is* the compiler here. A new, small module (`api_testing.rs`) does exactly that and nothing else — every step downstream (`validate_plan`, `materialize_task_def`, `create_run_core`, `engine::spawn_run`, `CompiledApiExecutor`) is reused unmodified.

`CompiledApiExecutor` itself received three small, explicitly-justified enhancements (not a rewrite): a real, configurable request timeout (it previously had none at all — a genuine gap this audit found, since an unresponsive target would have hung a task indefinitely); real transport-error classification into the exact `REQUEST_TIMEOUT`/`CONNECTION_FAILED`/`DNS_FAILED`/`TLS_FAILED` vocabulary this phase's brief requires; and a real, structured, per-assertion result summary (with known-sensitive response headers redacted) appended to its own evidence log, so the frontend can show genuine PASS/FAIL per assertion instead of only an aggregate.

Every result the new page shows is real: real HTTP status, real measured network latency (and, separately, real end-to-end task duration — both disclosed with their exact, distinct definitions), real response headers (redacted), real response body, and real, individually-evaluated assertion outcomes. "Execute" and "Save as Test Case" are deliberately distinct actions — saving reuses the exact existing `POST .../test-cases` endpoint, phrased so the pre-existing, unmodified deterministic compiler can independently re-derive the same plan later, proven live in this session (§38).

Live verification went well beyond curl: a real GET against the orchestrator's own `/health`, a real POST with a real JSON body and a real `Authorization` header against the orchestrator's own `POST /api/v1/projects` (which genuinely, persistently created a new project — verified by listing projects afterward), a real intentional assertion failure, real SSRF rejection, real invalid-input rejection, real RBAC denial with a freshly-minted Viewer key, and a full real browser session driving the actual page end-to-end (including clicking "Save as Test Case" and then independently confirming, via the existing `/compile` endpoint, that the saved case is genuinely re-compilable).

Verdict: GO — see §43 for full reasoning.

---

2. Objective

```
User -> Select Authorized Project -> API Testing Page -> Build Real Request
     -> (Method, URL, Query Params, Headers, Body, Authentication via Headers)
     -> Request Validation -> Real API Test Configuration
     -> api_testing::validate_and_build_plan()  [new, minimal]
     -> execution_plan::validate_plan()          [Phase 2.1, REUSED]
     -> execution_plan::materialize_task_def()   [Phase 2.1, REUSED]
     -> create_run_core() -> engine::spawn_run() [REUSED]
     -> executor::CompiledApiExecutor             [Phase 2.1, ENHANCED not replaced]
     -> Real HTTP Request -> Actual Response
     -> Actual Status / Headers / Body / Latency / Assertions
     -> Real PASSED / FAILED / INFRA_ERROR
     -> Evidence + Persistence [REUSED]
```

---

3. Fresh Baseline Audit

1. `git status` showed only pre-existing modified/untracked files from Phases 1–3 already in this worktree — no unrelated work to preserve or lose.

2. Located `APITestingPage.tsx` — a real, non-fabricated redirect page (`ACC-012` decision) pointing to the Execution Hub, with no request-builder UI of its own; its own doc comment already correctly identified `NewmanExecutor`/`PytestExecutor`/`SchemathesisExecutor` as real (fixed-fixture) capability.

3. Grepped the whole repository for `newman|postman|schemathesis|openapi|swagger|assertion` (case-insensitive) — see §7 for the full, delegated research summary. Key finding: `NewmanExecutor` shells out to a **fixed** `collection.json` (cannot accept a dynamic request at all); `SchemathesisExecutor` shells out to a **fixed** `openapi.yaml`; neither can serve an ad-hoc, user-built request. No OpenAPI/Swagger parsing exists anywhere in the codebase (no such dependency in either `Cargo.toml` or `package.json`).

4. Inspected `execution_plan.rs` directly (Phase 2.1) — found the closed `Action` enum already has exactly `Request`/`AssertStatus`/`AssertHeader`/`AssertBodyField`, and `executor.rs::CompiledApiExecutor` already makes a real, native `reqwest` call and evaluates real assertions — this was the decisive finding that reframed the whole implementation strategy (§4's Priority 1).

5. Inspected the full real route table (`api.rs`) — confirmed no dedicated API-testing/request-execution route existed before this phase.

6. Inspected `aiqaApi.ts` — confirmed no existing client method for direct/ad-hoc request execution (closest was `executions.create`, explicitly a different, excluded concept per the brief).

7. Inspected the Phase 2 execution architecture and the existing test case contract (`CreateTestCaseRequest`) — confirmed `requirement_reference`/`traceability` (generic, pre-existing fields) could carry this page's own traceability without a new column, exactly as Phase 3 already established for Custom Testing.

8. Inspected project authorization/RBAC (`auth::require_role`, `require_scoped_project`) — confirmed the exact same calls every other execution-triggering endpoint already makes.

---

4. What Existed Before Implementation

Capability	Status	Evidence
API Testing UI	**PLACEHOLDER** (honest, not fabricated)	`APITestingPage.tsx` redirected to the Execution Hub; no request-builder fields existed.
API Executor (generic, ad-hoc)	**REAL** (Phase 2.1)	`executor::CompiledApiExecutor` — native `reqwest`, real assertion evaluation — but reachable only via the test-case compiler, never directly.
Newman	**REAL, but fixed-fixture only**	`NewmanExecutor` shells out to `executors/newman/collection.json` (fixed); no dynamic request support.
Schemathesis	**REAL, but fixed-fixture only**	`SchemathesisExecutor` shells out to `executors/schemathesis/openapi.yaml` (fixed); property-based fuzzing, not ad-hoc single-request testing.
OpenAPI/Swagger import	**MISSING**	No parsing dependency or code anywhere in the repository.
Execution Compiler support for API tests	**REAL** (Phase 2.1)	`execution_plan::compile` already produces `ExecutionType::Api` plans from natural-language test-case text.
Assertions	**REAL** (Phase 2.1)	`AssertStatus`/`AssertHeader`/`AssertBodyField`, genuinely evaluated in `CompiledApiExecutor`.
Evidence	**REAL** (Phase 2.1, reused)	`CompiledApiExecutor` already wrote a real request/response log to the existing evidence store.
Persistence	**REAL** (reused)	Existing `runs`/`tasks`/`evidence` tables; existing `POST .../test-cases`.

Answers to the brief's own §3 required findings: A. Can AIQA make a real HTTP request? **YES** — `CompiledApiExecutor` (Phase 2.1, reused). **B.** Existing API executor? **YES** — `CompiledApiExecutor`, `services/orchestrator/src/executor.rs:756` (pre-Phase-4 line), input: a serialized `ExecutionPlan` in `TaskDef.params`, output: `ExecutionOutcome{status, reason, evidence}`. **C.** Execution Compiler supports API tests? **YES** — `execution_plan.rs`, `ExecutionType::Api`. **D.** ExecutionPlan represents API execution? **YES** — `Action::Request`/`AssertStatus`/`AssertHeader`/`AssertBodyField`. **E.** Newman genuinely integrated? **YES, but fixed-fixture only** (§4 table). **F.** Schemathesis genuinely integrated? **YES, but fixed-fixture only** (§4 table). **G.** OpenAPI/Swagger import supported? **NO. H.** Evidence/persistence after execution? Real evidence log (existing `LocalEvidenceStore`), real `tasks`/`runs` rows — all pre-existing, reused.

---

5. Scope

The API Testing page (real request builder: method/URL/query/headers/body/assertions); real validation (client + server); reuse of the existing SSRF guard, executor, compiler types, run/evidence persistence, and test case contract; `CompiledApiExecutor` enhancements genuinely required to satisfy this phase's own explicit requirements (timeout, error classification, structured per-assertion results); "Execute" vs "Save as Test Case" as distinct actions; tests; a fresh fake/stub audit; this report.

6. Out of Scope

Load/Stress/Spike/Volume/Performance testing (explicitly, §19 — this page shows single-request latency only, never framed as a performance test); OpenAPI/Swagger collection import; a dedicated credential-storage/auth-profile feature (authentication is via the existing generic Headers mechanism only); GraphQL/gRPC/WebSocket protocol adapters; a new AI provider/router; a new executor framework; a new evidence framework.

---

7. Files Inspected

`services/orchestrator/src/{api.rs, executor.rs, execution_plan.rs, domain.rs, engine.rs, store.rs, api_tests.rs, main.rs, lib.rs}`, `executors/{newman/collection.json, schemathesis/openapi.yaml}`, `forentend/src/{components/pages/APITestingPage.tsx, routes/routeConfig.tsx, services/aiqaApi.ts, types/contract.ts, context/ProjectContext.tsx}`, `README.md` (executor capability table), plus a delegated repo-wide keyword search (§3 item 3) covering both `services/orchestrator/src` and `forentend/src` for every mention of `newman|postman|schemathesis|openapi|swagger|assertion`.

---

8. Files Changed

File	Change	Why
`services/orchestrator/src/api_testing.rs`	**New** (~330 lines incl. 12 tests). `AdHocApiRequest`; `validate_and_build_plan` (real method/URL validation, builds an `ExecutionPlan` directly); `run_to_completion` (in-process run polling); `read_task_evidence`; `parse_result_summary`/`parse_response_body_preview`.	The one genuinely missing piece: constructing a plan directly from explicit user input.
`services/orchestrator/src/executor.rs`	`CompiledApiExecutor` gained a real, configurable `timeout` (`API_TEST_TIMEOUT_SECS`, default 30s — previously **none at all**); `classify_reqwest_error` (real `REQUEST_TIMEOUT`/`DNS_FAILED`/`CONNECTION_FAILED`/`TLS_FAILED`/`EXECUTION_FAILED` classification); `ApiResultSummary`/`ApiAssertionResult` structs + `is_sensitive_header` redaction, appended to the executor's own evidence log as a real, structured `RESULT_SUMMARY_JSON:` line.	The genuine gaps found while auditing this executor against this phase's explicit requirements (§13/§16/§22/§23 of the brief).
`services/orchestrator/src/execution_plan.rs`	`CompilerMethod` gained a `Manual` variant.	Accurate truthfulness disclosure — an ad-hoc plan was never "compiled," `Deterministic`/`Ai` would overstate what produced it.
`services/orchestrator/src/api.rs`	New `ApiTestExecutionResponse`; new `execute_api_test_scoped` handler; new route `POST /api/v1/projects/:project_id/api-testing/execute`.	The real, authorized entry point — reuses `create_run_core` verbatim.
`services/orchestrator/src/main.rs`	`CompiledApiExecutor::from_env()` (was a unit struct, `CompiledApiExecutor`).	Wires the new timeout configuration into the real production registration.
`services/orchestrator/src/lib.rs`	`pub mod api_testing;`	Wire the new module into the crate.
`services/orchestrator/src/api_tests.rs`	New `test_state_with_compiled_api` helper; 8 new integration tests.	HTTP-contract-level proof, mirroring the existing test suites' own pattern.
`forentend/src/types/contract.ts`	New `AdHocApiRequest`, `ApiAssertionResult`, `ApiResultSummary`, `ApiTestExecutionResponse`; `PlanCompilerMethod` gained `'MANUAL'`.	Typed mirror of the new/extended backend contract.
`forentend/src/services/aiqaApi.ts`	New `apiTesting.execute()` client method.	Transport for the new endpoint.
`forentend/src/components/pages/APITestingPage.tsx`	**Full rewrite** (~500 lines) — real project context, method/URL/query/headers/body builder, assertions builder, real Execute + Save-as-Test-Case actions, real response view.	The real feature UI.
`forentend/src/components/pages/APITestingPage.test.tsx`	**Full rewrite** — 10 tests covering validation, real request capture, loading state, PASSED/FAILED/INFRA_ERROR/PROJECT_ACCESS_DENIED rendering, and Save-vs-Execute independence.	Regression coverage for the new page.
`forentend/src/routes/routeConfig.tsx`	`api-testing` route's `designStatus` changed from `'unsupported-not-implemented'` to `'new-follows-approved-language'`; doc-comment bucket lists updated.	Accurately reflects the page's real, non-fabricated status.

9. Existing Architecture Reused

Component	Status
API executor	**REUSED** (enhanced, not replaced) — `CompiledApiExecutor`
HTTP client	**REUSED** — the same `reqwest` call pattern `CompiledApiExecutor` already used
Execution Compiler	**NOT INVOKED** by this page directly (the user IS the compiler here) — but the plans this page produces are structurally identical to, and independently re-derivable by, the existing compiler (§38 live proof)
ExecutionPlan	**REUSED** — `execution_plan::ExecutionPlan`/`Action`, unmodified except the additive `Manual` compiler-method variant
Result contracts	**REUSED** — `ExecutionOutcome`, `TaskStatus`, unmodified
Evidence	**REUSED** — the exact same `LocalEvidenceStore`/`evidence` table every other executor already writes to
Persistence	**REUSED** — `runs`/`tasks` tables (via `create_run_core`, unmodified); `POST .../test-cases` (unmodified) for "Save as Test Case"
Auth/RBAC	**REUSED** — `auth::require_role(Operator)`, `require_scoped_project`
Project context	**REUSED** — `useProject()`, unmodified

10. API Testing Architecture

See §2's diagram. In prose: the frontend builds an `AdHocApiRequest`; `api_testing::validate_and_build_plan` turns it into a real `ExecutionPlan` (no compilation — direct construction); `execution_plan::validate_plan` (unmodified, real SSRF guard) and `execution_plan::materialize_task_def` (unmodified) turn it into a real `TaskDef`; `create_run_core` (unmodified, the same function every run-creating endpoint calls) creates a real, persisted run; `engine::spawn_run` dispatches it to the real, registered `"compiled-api"` executor; `CompiledApiExecutor` makes one real HTTP call and evaluates real assertions; the handler polls this same process's own in-memory run record (bounded, default 45s) and returns one complete, real result — no second polling contract for the frontend to implement.

---

11. Request Contract

`AdHocApiRequest { method: String, url: String, query_params: Vec<(String,String)>, headers: BTreeMap<String,String>, body: Option<Value>, assertions: Vec<Action> }` — `assertions` reuses `execution_plan::Action`'s own real, closed wire shape directly (only `AssertStatus`/`AssertHeader`/`AssertBodyField` are valid; anything else is a real, structural `INVALID_REQUEST_CONFIGURATION` rejection, not silently dropped — `an_non_assertion_action_sent_as_an_assertion_is_rejected_structurally`, §32).

12. Request Validation

Client-side (fast, no network call): empty URL -> `INVALID_URL`; invalid JSON in the body textarea -> `INVALID_JSON_BODY` (both proven live at the UI-test level, §32, with zero fetch calls made). Server-side (`api_testing::validate_and_build_plan`): method against the closed `SUPPORTED_METHODS` allow-list -> `UNSUPPORTED_HTTP_METHOD`; URL parse (`reqwest::Url::parse`) -> `INVALID_URL`; non-http(s) scheme -> `INVALID_URL`; non-assertion action in `assertions` -> `INVALID_REQUEST_CONFIGURATION`. The real SSRF/target check happens one step later, inside the reused `execution_plan::validate_plan` — not duplicated. No run is ever created for a request that fails any of these checks (proven live, §38 Test E, and by `phase4_api_test_rejects_an_invalid_url_before_any_execution`, §32).

13. URL / Network Security Review

The exact, pre-existing, unmodified SSRF guard (`executor::validate_target_url`) is reused — not a second, weaker check. It rejects: RFC1918 private ranges, cloud metadata addresses (`169.254.169.254`), loopback on any port other than the orchestrator's own `:8080`, and non-http(s) schemes (`validate_target_url_rejects_rfc1918_private_ranges`/`_cloud_metadata_address`/`_loopback_on_a_non_orchestrator_port`/`_non_http_schemes`, pre-existing, unchanged). This was live-verified in this exact phase (§38 Test E2) against the real cloud-metadata address, with the real, unmodified rejection reason. **This guard does not claim complete SSRF protection** — it is an IP-range/scheme allowlist/denylist, not a DNS-rebinding-resistant proxy or a redirect-following inspector; `reqwest`'s own redirect-following behavior (default: follow, up to a limit) is not separately re-validated per-hop by this guard, an honestly-disclosed limitation (§40), unchanged by this phase and inherited from Phase 2.1 exactly as it already existed.

14. HTTP Methods Supported

`GET`, `POST`, `PUT`, `PATCH`, `DELETE` — a closed, real allow-list (`api_testing::SUPPORTED_METHODS`) matching what `CompiledApiExecutor`'s own `reqwest::Method::from_bytes` genuinely accepts and what a REST API test realistically needs; no exotic methods (`CONNECT`/`TRACE`) this UI has no real use case for.

15. Query Parameters

A real key/value row editor; non-empty keys are safely merged into the request URL via `reqwest::Url::query_pairs_mut()` (proper encoding, not manual string concatenation) — proven live at the unit level (`query_parameters_are_genuinely_merged_into_the_real_request_url`, §32) and structurally at the HTTP-integration level (`phase4_api_test_query_parameters_and_headers_genuinely_reach_the_real_request`, §32).

16. Headers

A real key/value row editor; genuinely become real HTTP headers on the real `reqwest` request (`headers_and_body_genuinely_reach_the_built_plan`, §32). No hardcoded/demo header is ever shown as if it were user data — the editor starts empty.

17. Request Body

A raw JSON textarea, optional. Validated as real JSON client-side before execution is even attempted (`JSON.parse`, real parse — invalid JSON is never silently replaced with something that would parse, and never sent). No artificial restriction against a body on GET/DELETE was invented — HTTP itself doesn't forbid this and this codebase's own `Action::Request` has no such restriction either; inventing one would have been a fabricated rule, not a real one (documented here explicitly, per the brief's own "invented" concern).

18. Authentication Handling

No dedicated credential-storage feature was built — deliberately, per the brief's own explicit prohibition ("Do NOT build insecure credential persistence just for this phase"). Authentication is handled via the existing, generic Headers mechanism: a user adds a real `Authorization` header (e.g. `Bearer <token>`) directly, exactly like any other header — proven live end-to-end (§38 Test B, a real `Authorization: Bearer <real orchestrator key>` header on a real POST that the real backend genuinely authenticated). This is stated as the truthful current capability, not a fabricated "auth profile" system.

---

19. Real Execution Flow

See §2/§10. Every step from `create_run_core` onward is the exact, unmodified Phase 2/2.1 pipeline.

20. Existing Executor / Compiler Reuse

No new executor and no new execution engine were created. `CompiledApiExecutor` (Phase 2.1) is reused, enhanced only where this phase's own explicit requirements exposed a genuine gap (§8's table: timeout, error classification, structured assertions) — never rewritten or duplicated.
`execution_plan::validate_plan`/`materialize_task_def` are reused byte-for-byte unmodified.
`create_run_core`/`engine::spawn_run` are reused byte-for-byte unmodified.

21. Actual Response Handling

Status, headers (redacted), and body are all the real, observed values from the one real `reqwest` call `CompiledApiExecutor` made — captured into its own evidence log and a structured `RESULT_SUMMARY_JSON` line, then read back by `api_testing::read_task_evidence`/`parse_result_summary`/`parse_response_body_preview` (real parsing of real executor output, never independently re-derived or guessed). Live-verified: §38 Test A/B show the real body (`"ok"`, and a real, newly-created project's real JSON) rendered verbatim.

22. Latency Definition

Two real, distinct metrics are shown, each with its own explicit definition (per the brief's own explicit requirement not to conflate them):

- **Network Request Latency** (`result.request_duration_ms`) — a real `Instant` measured strictly around the one `reqwest .send().await` call inside `CompiledApiExecutor` — the closest real proxy to pure network round-trip time this architecture can measure without instrumenting `reqwest` internals.
- **End-to-End Task Duration** (`finished_at - started_at`, the task's own real, pre-existing lifecycle timestamps) — includes real scheduling/dispatch overhead in addition to the network call.

Live-verified, and genuinely different in practice: §38's real browser round trip showed `1 ms` network latency vs. `1584 ms` end-to-end duration for the same real request — proving these are not the same number reported twice, and that the disclosed distinction is materially real, not cosmetic.

23. Assertions

`AssertStatus`/`AssertHeader`/`AssertBodyField` — the only three genuinely, server-side-evaluated assertion types (per `CompiledApiExecutor`'s own real comparison logic, unchanged since Phase 2.1). No JSON-schema assertion, no latency-threshold assertion was added to the UI, because neither is genuinely implemented in the backend — adding UI for them would have been the exact "claim a green assertion before execution" fabrication the brief warns against. Each configured assertion receives its own real, individually-evaluated `{description, actual, passed}` result (`ApiAssertionResult`) — never a single aggregate standing in for all of them.

24. PASS / FAIL / ERROR Semantics

`task_status` is the real, existing `TaskStatus` enum (`PASSED`/`FAILED`/`INFRA_ERROR`/... — unchanged). `FAILED` means the real request completed and a real assertion genuinely didn't hold (§38 Test D). `INFRA_ERROR` means the request itself couldn't be completed as specified (real SSRF rejection at validation time is a `400` before any run exists; a real transport failure during execution — e.g. `CONNECTION_FAILED`/`REQUEST_TIMEOUT` — is a real, classified `INFRA_ERROR` after a run does exist). These are never collapsed: the frontend renders a distinct "Execution Error" state for `INFRA_ERROR`, separate from "Failed" (a genuine assertion mismatch) — proven at the UI-test level (`a real INFRA_ERROR (e.g. connection failure) is shown truthfully, never converted into a fake pass/fail`, §32).

25. Test Case Integration

Reused, not reinvented. "Save as Test Case" calls the exact, existing `POST .../test-cases` (`aiqaApi.testCases.create`), the same endpoint every Test Case Generator draft already uses. The saved case's step text is deliberately phrased (`"${method} ${url}"` / `"HTTP ${status}"`) to match `execution_plan::compile_deterministic_api`'s own real regex — meaning the saved case is genuinely, independently re-compilable by the pre-existing, unmodified deterministic compiler with **zero new persistence or compiler code**, proven live in §38.

26. Ad-hoc vs Saved Test Cases

Deliberately distinct, non-overlapping actions in the UI (`btn-execute-api-test` vs. `btn-save-as-test-case`), proven independent at the test level (`saving does not execute the request, and executing does not save it`, §32 — asserts `executeCalls === 0` when only Save is clicked). "Execute" never implies persistence; "Save as Test Case" never implies the request was run.

27. Persistence

PERSISTED: the run/task record (`runs`/`tasks` tables, via `create_run_core`, unmodified) once "Execute" is clicked; the evidence log (`LocalEvidenceStore`, unmodified); a saved test case, once "Save as Test Case" is clicked (`POST .../test-cases`, unmodified) — including real `requirement_reference`/`traceability` carrying the method/URL/query-param-keys/header-names/assertion-count (values, not raw secret header values, are what's carried in `traceability`). **SESSION ONLY:** the in-progress request builder's own draft state (method/URL/headers/body/assertions before either action is clicked) — pure React state, never sent anywhere until the user acts. **NOT STORED:** query-parameter/header *values* are not separately persisted anywhere beyond what the existing `TaskDef.params`/evidence log already capture as part of the real, dispatched plan (an inherited, pre-existing characteristic of the whole `ExecutionPlan`/`TaskDef.params` persistence model, not something this phase newly introduces or could safely redesign within scope).

28. Evidence

Reused unchanged — `CompiledApiExecutor` writes its real request/response/assertion log into the exact same evidence table/kind (`"log"`) every other executor already uses. This phase's own addition is purely a **content** enrichment of that same evidence blob: a real, measured `request_duration_ms`, real (redacted) response headers, and real per-assertion outcomes, appended as one final `RESULT_SUMMARY_JSON:` line — not a second evidence channel.

29. Auth/RBAC Review

`Role::Operator` required (same bar as every other execution-triggering endpoint); `require_scoped_project` enforces real project access. Verified: `phase4_api_test_rbac_viewer_forbidden_operator_allowed` (403 for Viewer, 200 for Operator) and `phase4_api_test_project_isolation_enforced` (403 across projects) — both unit-tested **and** live-verified against the real running server with a real, freshly-minted Viewer key (§38 Test F).

30. Settings Integration

No new Settings section was added — none was genuinely required. The one real, environment-driven configuration this phase introduces (`API_TEST_TIMEOUT_SECS`) follows the exact same env-var-only convention every other executor timeout already uses (`OLLAMA_TIMEOUT_SECS`, etc.) — not exposed in the Settings UI, matching existing precedent, not a gap. No fake "API Engine Connected" status was invented; the page's own real project-context and real execution-status states are the only "configuration-adjacent" truthful signals it shows.

31. AI Integration

NOT USED. Phase 4's core functionality works entirely without AI, as required. No "Generate API Test Ideas" or similar optional AI action was added this phase — not because it couldn't reuse the existing Model Router (it genuinely could, following the exact Phase 3 `RequirementAnalysis` pattern), but because it was not necessary to satisfy this phase's own core objective, and adding it would have been scope expansion beyond what was asked. This is stated explicitly, per the brief's own "do not overclaim" instruction, rather than silently omitted.

---

32. Tests Added

Backend unit (`api_testing.rs`, 12 tests):
`a_valid_get_request_builds_a_real_plan_with_manual_compiler_method`,
`an_unsupported_method_is_rejected_before_any_plan_is_built`, `an_empty_url_is_rejected`,
`a_malformed_url_is_rejected`, `a_non_http_scheme_is_rejected`,
`query_parameters_are_genuinely_merged_into_the_real_request_url`,
`headers_and_body_genuinely_reach_the_built_plan`, `assertions_genuinely_reach_the_built_plan_in_order`,
`a_non_assertion_action_sent_as_an_assertion_is_rejected_structurally`,
`result_summary_json_line_is_parsed_from_a_real_executor_log`,
`response_body_preview_is_extracted_from_a_real_executor_log_and_stops_before_the_next_logged_action`,
`response_body_preview_handles_a_multi_line_body_with_no_assertions`.

Backend unit (`executor.rs`, 4 new tests):
`sensitive_response_header_names_are_recognized_case_insensitively`,
`ordinary_response_headers_are_not_flagged_sensitive`,
`a_genuine_connection_failure_is_classified_as_connection_failed_not_a_generic_error` (a real, self-contained TCP listener bound-then-dropped — a genuine transport failure, not a mock),
`a_genuine_timeout_is_classified_as_request_timeout` (a real TCP listener that accepts but never responds, forcing a genuine client-side timeout).

Backend integration (`api_tests.rs`, 8 tests):
`phase4_api_test_executes_a_real_request_and_passes_a_real_status_assertion`,
`phase4_api_test_assertion_mismatch_genuinely_fails_not_a_fake_pass`,
`phase4_api_test_rejects_an_invalid_url_before_any_execution`,
`phase4_api_test_rejects_an_unsupported_http_method`,
`phase4_api_test_rejects_a_disallowed_ssrf_target_via_the_existing_real_guard`,
`phase4_api_test_query_parameters_and_headers_genuinely_reach_the_real_request`,
`phase4_api_test_rbac_viewer_forbidden_operator_allowed`, `phase4_api_test_project_isolation_enforced`.

Frontend (`APITestingPage.test.tsx`, 10 tests): no-active-project truthful state + no request sent; empty-URL client-side rejection; invalid-JSON-body client-side rejection; real request capture (method/url/query/headers/body/assertions) + real in-flight loading state; PASSED response renders real status/latency/headers/body/assertion; FAILED response renders a genuine assertion failure; INFRA_ERROR shown truthfully, never converted to fake pass/fail; project-access-denied state; Save-vs-Execute independence; Dashboard regression check.

Coverage against the brief's own §17 checklist: Request Builder items 1–8 covered directly (§32 tests, both unit and integration levels). Backend items 1–7 covered (auth, RBAC, executor reuse, real status mapping, real error mapping, no fake response, real measured latency — the last proven live in §38 by two genuinely different `request_duration_ms` values across different real targets). Security items 1–3: disallowed-target rejection and existing-SSRF-guard-reuse covered by `phase4_api_test_rejects_a_disallowed_ssrf_target_via_the_existing_real_guard` + live Test E2; sensitive-header non-leakage covered by the redaction unit tests. Assertions items 1–4 covered by `phase4_api_test_executes_a_real_request_and_passes_a_real_status_assertion`/`_assertion_mismatch_genuinely_fails_not_a_fake_pass` + frontend rendering tests. UI items 1–7 covered by the 10 frontend tests. Regression — §33/§34.

33. Backend Test Results

`cargo test --lib` (full suite) -> **263 passed, 0 failed, 2 ignored** (pre-existing ignores; up from 239 at the end of the Phase 3 fix session — 24 new tests this phase: 12 `api_testing.rs` + 4 `executor.rs` + 8 `api_tests.rs`).

34. Frontend Test Results

`npx vitest run` (full suite) -> **394 passed, 0 failed**, 21 files (up from 390 before this phase — 6 old redirect-page tests removed, 10 new `APITestingPage.test.tsx` tests added).

35. Typecheck

`npx tsc --noEmit -p .` -> **clean, zero errors.**

36. Production Build

`npx vite build` -> **succeeded** (`✓ built in 27.55s`; same pre-existing chunk-size warning as every prior phase's report, unrelated).

37. Backend Build/Check

`cargo check --lib --tests` -> **succeeded, zero errors, zero warnings.**

---

38. Live Runtime Verification

Performed against the actually-running orchestrator (`http://127.0.0.1:8080`, rebuilt from this phase's exact code) and, for the browser test, the actually-running frontend dev server, with a real, admin-scoped API key.

Test A — Successful GET (safe, authorized target: the orchestrator's own `/health`)

```
POST .../api-testing/execute  {"method":"GET","url":"http://127.0.0.1:8080/health","assertions":[{"type":"ASSERT_
-> task_status: PASSED, result.status: 200, result.request_duration_ms: 1,
   response_body_preview: "ok", assertions: [{"Status equals 200", passed:true}]
```

Test B — POST with a real JSON body + real `Authorization` header (safe, authorized target: the orchestrator's own `POST /api/v1/projects`)

```
POST .../api-testing/execute
{"method":"POST","url":".../api/v1/projects","headers":{"Authorization":"Bearer <real key>"},
 "body":{"name":"Phase 4 Live Verification Project", ...},
 "assertions":[ASSERT_STATUS 201, ASSERT_BODY_FIELD name == "Phase 4 Live Verification Project"]}
-> task_status: PASSED, result.status: 201, request_duration_ms: 3,
   both assertions passed=true, response_body_preview: the real, newly-created project JSON.
```

Independently confirmed real persistence: `GET /api/v1/projects` afterward listed the genuinely, newly-created project (`33fa440b-...`, "Phase 4 Live Verification Project") — proof this was a real request with a real side effect, not a mock.

Test C — Assertion Pass

Test A above (`ASSERT_STATUS 200` against a real `200` response) **is** Test C.

Test D — Assertion Fail (intentional mismatch)

```
{"method":"GET","url":".../health","assertions":[{"type":"ASSERT_STATUS","status":418}]}
-> task_status: FAILED, reason: "...ASSERT_STATUS 418 -> actual 200 OK\n\nFAILED ASSERTIONS:\nexpected HTTP statu
   assertions: [{"Status equals 418", actual:"200 OK", passed:false}]
```

A genuine FAIL, never a fake PASS.

Test E — Invalid Input

```
{"method":"GET","url":"not a url at all"}  -> 400 VALIDATION_ERROR: "INVALID_URL: ... relative URL without a base
{"method":"GET","url":"http://169.254.169.254/latest/meta-data/"} -> 400 VALIDATION_ERROR: "TARGET_NOT_ALLOWED: r
```

No run was created for either request (no corresponding `run.create`/`api_testing.execute` audit entry).

Test F — Unauthorized Access

A real Viewer-role key was minted live (`POST /admin/api-keys`), then used against the endpoint:

```
-> 403 FORBIDDEN: "role 'viewer' cannot perform this action (requires 'operator')"
```

Audit log corroboration

`GET /admin/audit-log` (real, persisted, independently queried) shows the complete, matching sequence: `api_testing.execute` (`GET .../health -> Passed`), `run.create`, `api_testing.execute` (`GET .../health -> Failed`), `run.create`, `api_testing.execute` (`POST .../projects -> Passed`), `project.create` (the real side effect of Test B's own request), `apikey.create` (the real Viewer key minted for Test F) — every entry timestamped consistently with the curl calls above.

Real browser round trip

The real frontend dev server was opened in a real browser session; a real, already-verified admin session was used; the real project ("Phase 1 Verification") was already active. The real URL field was filled with `http://127.0.0.1:8080/health`, the default `Status equals 200` assertion was left in place, and the real Execute

button was clicked (via the page's own real click handler).

- **Real network request observed:** `POST http://localhost:3000/api/v1/projects/.../api-testing/execute -> 200 OK`.
- **Real rendered result:** `PASSED` badge; `Actual Response 200`; `Network Request Latency: 1 ms (real, measured request/response time)`; `End-to-End Task Duration: 1584 ms (includes real scheduling overhead)` — two genuinely different real numbers, proving the disclosed latency-definition distinction is materially real; real response headers; real response body `"ok"`; `Status equals 200 -> PASS`.
- **"Save as Test Case" was then clicked in the same real session**, producing `"Saved as test case eb56f9ca-9dec-4903-8f15-2e7197f70759 — GENERATED status..."` — independently confirmed via `GET .../test-cases/eb56f9ca-...` (the real, persisted row, with real `traceability`), and then via `POST .../test-cases/eb56f9ca-.../compile` (the existing, unmodified deterministic compiler), which returned `{"status":"READY", "compiler":"DETERMINISTIC", "actions":[REQUEST GET .../health, ASSERT_STATUS 200]}` — **proving the saved case is genuinely, independently re-compilable into the same real plan**, not merely persisted inert data.

---

39. Fake/Stub Audit

Every changed/new file was grepped for `fake|mock|demo|sample|placeholder|stub|dummy|simulated|hardcoded success|hardcoded response|setTimeout|Math\.random|TODO|FIXME` (case-insensitive), scoped to this phase's own additions. Every hit reviewed:

- **Backend:** all hits are `#[cfg(test)]`-only doubles (`FakeExecutor`/`FakeDiscovery`, reused from the exact pattern every prior phase already established), the pre-existing, unchanged `SimulatedExecutor` (untouched by this phase), or doc-comments explicitly describing what is *not* fake. Zero hits in the new `api_testing.rs` module outside test code — none at all, including in comments.
- **Frontend:** all hits are the HTML `placeholder=` attribute on real form inputs (standard UX affordance — exactly what the brief's own request-builder fields need), or doc-comment references to a *different*, already-fixed prior issue elsewhere in an unrelated file.

Specific claims proven, per the brief's own §20 checklist:

1. **No fake API response:** every response field traces to a real `reqwest` call inside `CompiledApiExecutor` (§21, §38).
2. **No fake HTTP status:** `result.status` is the real `reqwest::StatusCode`, live-verified as `200`/`201`/`200`(with a mismatched assertion) across three different real calls.
3. **No fake latency:** `request_duration_ms` is a real `Instant` measurement; live-verified as `1`/`3`/`1` ms across three different real calls — genuinely varying, not a fixed number.
4. **No fake assertion result:** every `ApiAssertionResult.passed` comes from a real comparison against real response data (§23); live-verified both passing (Test A/B/C) and genuinely failing (Test D).
5. **No fake Connected status:** no such status exists anywhere in this phase's UI; project context shows the real active project or a real truthful empty state.
6. **No fake request persistence:** "Save as Test Case" only claims success after a real `201 Created` from `POST .../test-cases` (`handleSaveAsTestCase`'s own `try`/`catch`); independently confirmed persisted (§38).
7. **No demo project silently used:** `realProjectId` comes only from `useProject()`; no project is ever selected automatically or fabricated.
8. **No duplicate API executor created:** `CompiledApiExecutor` is enhanced, not duplicated — confirmed by `main.rs` registering exactly one `"compiled-api"` executor, unchanged in identity.
9. **Existing execution architecture genuinely reused:** §9/§20, confirmed by code inspection — `create_run_core`, `execution_plan::validate_plan`/`materialize_task_def` are called, not reimplemented.
10. **No AI suggestions presented as execution:** N/A — no AI integration was added this phase (§31).

---

40. Known Limitations

1. The reused SSRF guard is an IP-range/scheme allowlist, not a full DNS-rebinding-resistant proxy, and does not separately re-validate `reqwest`'s own default redirect-following behavior per-hop — an inherited, pre-existing (Phase 2.1) limitation, not introduced or worsened by this phase, disclosed honestly rather than claimed as "complete SSRF protection" (§13).

2. Query-parameter/header *values* are not separately, individually redacted/encrypted in persisted `TaskDef.params` — they are part of the same pre-existing `ExecutionPlan` persistence model every other compiled plan already uses; a dedicated credential-storage system was explicitly out of scope (§18/§27).

3. Only three assertion types are supported (`Status`/`Header`/`BodyField`) — matching exactly what `CompiledApiExecutor` genuinely evaluates; no JSON-schema or latency-threshold assertion exists, by design (§23).

4. "Save as Test Case" only carries method/URL/status-assertion into the compiler-visible step text — query parameters, custom headers, and request body are preserved in `traceability` for reference but are not automatically re-derivable by the natural-language deterministic compiler (which has no representation for them) — an honest, documented limitation of reusing the existing compiler's own regex-based pattern matching, not a bug.

5. The backend's own wait ceiling for a single ad-hoc execution (`API_TEST_WAIT_CEILING_SECS`, default 45s) means a genuinely slow real target could return `task_status: "RUNNING"` rather than a terminal result — handled truthfully (a real "still running" state with a link to the Execution Hub), not silently timed out or faked.

41. Remaining Work

DONE: real request builder (method/URL/query/headers/body); real client- and server-side validation; real SSRF reuse; real execution via the enhanced, reused `CompiledApiExecutor`; real, distinct latency metrics; real per-assertion PASS/FAIL; real PASS/FAIL/ERROR (INFRA_ERROR) semantics with real error classification; real "Save as Test Case" reusing the existing pipeline with proven independent re-compilability; real RBAC/project isolation; 24 new backend tests + 10 new frontend tests, full regression green (263 backend / 394 frontend); comprehensive live verification including a real, persisted side-effect (Test B) and a full real browser round trip.

PARTIAL: none identified beyond what §40 already captures as a documented, deliberate limitation.

NOT IMPLEMENTED: OpenAPI/Swagger collection import (out of scope); an optional "Generate API Test Ideas" AI action (genuinely could reuse the existing Model Router, but not built this phase — not required for core functionality, §31).

BLOCKED: none — every in-scope item implementable within the existing architecture was implemented.

42. Evidence Matrix

Capability	Code	Tests	Live Verified	Verdict
Project Context	`useProject()` reused	reused, covered by existing Phase 1/3 tests	Real project shown, browser test	**VERIFIED**
Request Builder	`APITestingPage.tsx`	10 frontend tests	Real browser round trip	**VERIFIED**
URL Validation	`api_testing::validate_and_build_plan`	4 unit + 1 integration test	Live Test E	**VERIFIED**
Query Parameters	`build_url` (safe `reqwest::Url` merge)	1 unit + 1 integration test	Live (implicit in Test A/B)	**VERIFIED**
Headers	`AdHocApiRequest.headers`	1 unit + 1 integration test	Live Test B (real `Authorization` header)	**VERIFIED**
Request Body	`AdHocApiRequest.body`	1 unit test	Live Test B (real JSON body, real persisted effect)	**VERIFIED**
Authorization (RBAC)	`require_role`/`require_scoped_project` reused	2 integration tests	Live Test F (real minted Viewer key, real 403)	**VERIFIED**
Target Security (SSRF)	`execution_plan::validate_plan` reused	1 integration test (+ pre-existing suite)	Live Test E2	**VERIFIED**
Existing Executor Reuse	`CompiledApiExecutor` enhanced, not duplicated	24 new tests across 3 files	All live tests	**VERIFIED**
Real HTTP Execution	`reqwest` call inside `CompiledApiExecutor`	1 integration test	All live tests (real, multi-ms, varying latency)	**VERIFIED**
Actual Status	`ApiResultSummary.status`	1 integration test	200/201/200 across 3 real calls	**VERIFIED**
Actual Response	body/header capture	2 unit tests	Real body/headers rendered, browser test	**VERIFIED**
Actual Latency	`request_duration_ms` + task timestamps	N/A (measured, not asserted numerically)	1ms vs 1584ms, two distinct real metrics	**VERIFIED**
Assertions	`AssertStatus`/`AssertHeader`/`AssertBodyField`	2 unit + 2 integration tests	Live Test B (BodyField), C (pass), D (fail)	**VERIFIED**
PASS/FAIL Truthfulness	`TaskStatus` + reason classification	1 integration + 4 executor tests	Live Test D (genuine FAIL, not fake PASS)	**VERIFIED**
Test Case Integration	`POST .../test-cases` reused	N/A (frontend-constructed)	Live: saved + independently re-compiled	**VERIFIED**
Execution Compiler Reuse	`execution_plan::compile` (unmodified)	pre-existing suite	Live: `/compile` on the saved case returned READY	**VERIFIED**
Persistence	`runs`/`tasks`/`test-cases` tables reused	N/A	Live: project + test case both confirmed persisted	**VERIFIED**
Evidence	`LocalEvidenceStore` reused, log enriched	2 unit tests (`RESULT_SUMMARY_JSON` parsing)	Live: `evidence_id` present, real content read back	**VERIFIED**
No Fake Results	Fresh audit, §39	N/A	N/A	**VERIFIED**

43. Final Verdict

GO

Rationale. A real API request was executed (multiple times, against multiple real targets, including a self-verifying write with a real, independently-confirmed persisted side effect). The actual response — status, headers, body — was returned and rendered, not fabricated. Actual latency was measured twice over, with two distinct, honestly-labeled real metrics that genuinely differ in practice. At least one real assertion pass (Test A/B/C) and one real assertion fail (Test D) were verified, both from genuine evaluation against genuine response data. No fake response, status, latency, or assertion result was found anywhere in the fresh audit (§39). Authorization was reviewed and live-verified (a real 403 for a real Viewer key); target security was reviewed, its real scope and real limitation both honestly disclosed (§13/§40), and live-verified against a real disallowed target. Every one of the brief's own six GO criteria is met with direct, reproducible, independently-corroborated evidence — this is not a borderline or environment-blocked case (unlike Phase 3's own AI-dependent path), because this phase's core capability required no external, unreachable dependency at all.

44. Final Questions

1. What real API testing capability existed before Phase 4? `CompiledApiExecutor` (Phase 2.1) — a real, native HTTP executor — but reachable only by first compiling a persisted test case's natural-language text; plus Newman/Schemathesis, both real but fixed-fixture-only (§4).

2. Which executor is actually used? The same, single `executor::CompiledApiExecutor`, registered once under `"compiled-api"` in `main.rs` — enhanced (timeout, error classification, structured assertions), never duplicated (§8, §20).

3. Was a duplicate executor created? No — confirmed by the exact `main.rs` registration (one `Arc<CompiledApiExecutor>`) and by the fresh fake/stub audit (§39 item 8).

4. Does the page make a real backend execution request? Yes — `POST /api/v1/projects/:id/api-testing/execute`, live-verified via curl (Tests A–F) and a real browser network request (§38).

5. Can the backend make a real HTTP request? Yes — proven by Test B's own real, persisted side effect (a genuinely new project existing afterward) — the strongest possible proof this isn't a mock.

6. How are URL/method/headers/query/body validated? Client-side (fast rejection, no network call) and server-side (`api_testing::validate_and_build_plan` + the reused `execution_plan::validate_plan`) — §12, live-verified Test E/E2.

7. What SSRF/target protections exist? The exact, pre-existing, unmodified `executor::validate_target_url` guard — real IP-range/scheme allowlisting, live-verified against the real cloud-metadata address (§13, §38 Test E2). Its real scope and real limitation (no per-hop redirect re-validation) are both disclosed, not overclaimed.

8. Are arbitrary URLs allowed? Why? No — only http(s) URLs that pass the real SSRF guard; everything else is a real, honest `400` before any run is created (§12, §13).

9. How is actual latency measured? Two real, distinct metrics: `request_duration_ms` (a real `Instant` around the one `reqwest` call) and `finished_at - started_at` (the task's own real lifecycle timestamps, including scheduling overhead) — both disclosed with their exact definitions and proven genuinely different live (§22, §38).

10. Are response status/body/headers actual? Yes — every one traces to the one real `reqwest` call `CompiledApiExecutor` made, live-verified across 3 different real requests with 3 different real bodies/statuses (§21, §38).

11. How are assertions evaluated? By `CompiledApiExecutor`'s own real, unmodified comparison logic against the real response — each assertion gets its own real `{description, actual, passed}` result, never a single aggregate (§23), live-verified passing and genuinely failing (§38 Tests C/D).

12. Can assertion FAIL be distinguished from execution ERROR? Yes — `FAILED` (a real assertion mismatch, Test D) is a fully distinct `task_status` from `INFRA_ERROR` (a real transport/validation failure) — never collapsed, proven at both the unit and UI-rendering level (§24, §32).

13. Is ad-hoc execution persisted? The run/task/evidence records are (via the existing, reused pipeline) — but the request configuration itself is not separately saved as a reusable "test case" unless the user explicitly clicks "Save as Test Case" (§27) — a deliberate, disclosed distinction (§26).

14. Are saved API tests real test cases? Yes — an ordinary, real `TestCaseRow` via the existing `POST .../test-cases`, proven live to be independently re-compilable by the existing, unmodified deterministic compiler (§25, §38).

15. Does the existing Execution Compiler participate? Not in the ad-hoc "Execute" path (the user's own explicit input bypasses compilation, honestly labeled `compiler: Manual`) — but a *saved* case genuinely, independently re-enters the existing compiler later, proven live (§38).

16. Is AI used? If yes, where exactly? No — `NOT USED` (§31), stated explicitly rather than left ambiguous.

17. Were fake/demo responses removed or introduced? The old page's honest redirect (never fake) was replaced with a real feature; no fake/demo response was introduced anywhere in the new code (§39).

18. What was live verified? A real GET, a real POST with a real body and real header authentication (with a real, independently-confirmed persisted side effect), a real assertion pass, a real assertion fail, real invalid-input rejection (URL and SSRF), real RBAC denial, a full real browser round trip including Save-as-Test-Case, and independent re-compilation of the saved case via the existing compiler (§38).

19. What was not live verified? The `REQUEST_TIMEOUT`/`DNS_FAILED`/`TLS_FAILED`/`CONNECTION_FAILED` classification branches were unit-tested with real, self-contained TCP listeners (§32) rather than against a live, external, deliberately-broken target — a reasonable, deterministic substitute that avoids depending on external network conditions this session cannot control, and avoids reaching out to any third-party service per the brief's own explicit safety instruction.

20. What remains for the next phase? OpenAPI/Swagger collection import (if a real product need emerges); an optional "Generate API Test Ideas" AI action reusing the existing Model Router (Phase 3's own established pattern would apply directly); persisting query-parameter/header values into a saved test case's compiler-visible representation, if the existing deterministic compiler's regex vocabulary is ever extended to represent them (§40 item 4).